

Git Guide

A Friendly Introduction for Beginners

Tailored for use with VS Code & Google Colab

■ Beginner Friendly

■ VS Code

■ Google Colab

■ GitHub Ready

This guide will take you from zero knowledge of Git to confidently tracking changes, collaborating on GitHub, and using Git every day inside VS Code and Google Colab.

Table of Contents

1. What is Git and Why Should I Care?
2. Core Concepts: The Mental Model
3. Installing Git
4. First-Time Setup
5. Your First Repository
6. The Daily Git Workflow
7. Branching – Working in Parallel
8. Working with GitHub (Remote Repos)
9. Git Inside VS Code
10. Git with Google Colab
11. Fixing Common Mistakes
12. Collaboration & Pull Requests
13. Quick-Reference Cheat Sheet

1. What is Git and Why Should I Care?

Git is a **version control system** – think of it as an unlimited Undo button for your entire project, combined with a time machine and a collaboration tool. Instead of having files named *report_final_v3_ACTUALLY_FINAL.ipynb*, Git tracks every change you make and lets you travel back to any point in history.

■ ■ Git ≠ GitHub. Git is the tool that runs on your computer. GitHub is a website that stores your Git history in the cloud so teammates can access it.

Why use Git?

- **Safety net** – you can always roll back a bad change.
- **History** – see exactly what changed, when, and why.
- **Teamwork** – multiple people can work on the same project without overwriting each other.
- **Industry standard** – every software company uses it.

2. Core Concepts: The Mental Model

Before touching the command line, it helps to understand the four places your code can live in Git's world:

Zone	What it is	Analogy
Working Directory	Files on your disk right now	<i>Your desk</i>
Staging Area	Changes you've marked 'ready to save'	<i>A tray on your desk</i>
Local Repository	Your full history saved on this computer	<i>Your personal filing cabinet</i>
Remote Repository	The copy on GitHub, shared with the team	<i>A shared office archive</i>

Key vocabulary

- **Repository (repo)** – the project folder that Git tracks.
- **Commit** – a snapshot of your project at a point in time. Like pressing Save + writing a note about what you changed.
- **Branch** – an independent line of development. You can experiment without touching the main code.
- **Merge** – combining two branches back together.
- **Clone** – downloading a remote repo to your computer.
- **Push** – uploading your commits to the remote (GitHub).
- **Pull** – downloading the latest commits from GitHub.

3. Installing Git

Windows

1. Go to <https://git-scm.com/download/win> and download the installer.
2. Run the installer. Leave all defaults – just click Next until Finish.
3. Open **Git Bash** (it was installed alongside Git) and type:

■ Verify installation

```
git --version
```

You should see something like *git version 2.44.0.windows.1*.

macOS

Open **Terminal** and run:

```
git --version
```

If Git is not installed, macOS will prompt you to install Xcode Command Line Tools – just click Install.

Linux (Ubuntu / Debian)

```
sudo apt update  
sudo apt install git
```

■ On Google Colab Git is already installed – you can skip this step entirely for Colab notebooks.

4. First-Time Setup

Git needs to know who you are so it can label your commits. Run these two commands once (replace with your real name and email):

■ Set your identity

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

■■ Use the same email address you registered with on GitHub – this links your commits to your profile.

Optional but recommended

Set VS Code as the default editor Git opens when it needs your input:

■ Set VS Code as Git editor

```
git config --global core.editor "code --wait"
```

Verify all your settings:

```
git config --list
```

5. Your First Repository

Option A – Start from scratch on your computer

■ Create a new local repo

```
mkdir my_project # create a folder
cd my_project # move into it
git init # turn it into a Git repo
```

Git creates a hidden **.git** folder inside – that is where all the history lives. Never delete it.

Option B – Clone an existing GitHub repo

If your project already exists on GitHub (the most common case when collaborating), copy its URL and run:

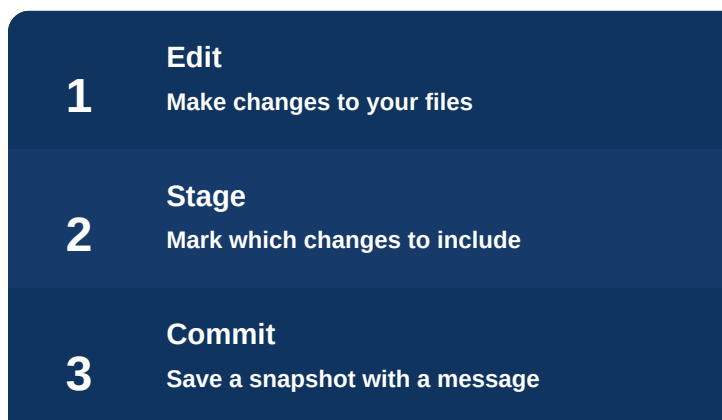
■ Clone from GitHub

```
git clone https://github.com/username/repo-name.git
cd repo-name
```

■ In VS Code you can also do this via the Command Palette: press Ctrl+Shift+P (or Cmd+Shift+P on Mac) and type 'Git: Clone'.

6. The Daily Git Workflow

Most of your day-to-day work follows the same three-step loop:



Check what has changed

■ See the state of your repo

```
git status
```

Red files = changed but not staged. Green files = staged, ready to commit.

Stage your changes

```
git add filename.py # stage one specific file
git add . # stage ALL changed files
```

Commit (save a snapshot)

■ Write a short, descriptive message

```
git commit -m "Add data cleaning function"
```

■ Write commit messages in the present tense and keep them short: 'Fix bug in preprocessing' not 'I fixed the thing'.

View your history

```
git log # full log
git log --oneline # compact one-line-per-commit view
```

7. Branching – Working in Parallel

A branch is like a copy of your project where you can try new ideas without affecting the main code. The default branch is called **main**.

■ Working with branches

```
git branch # list all branches
git branch feature/data-analysis # create a new branch
git checkout feature/data-analysis # switch to it

# Shortcut: create AND switch in one command
git checkout -b feature/data-analysis
```

Now do your work, stage, and commit as usual. When ready, merge it back to main:

■ Merging

```
git checkout main # switch back to main
git merge feature/data-analysis # bring in the changes
```

■■ Always commit or stash your work before switching branches – Git will warn you if you have unsaved changes.

Deleting a finished branch

```
git branch -d feature/data-analysis # safe delete (already merged)
```

8. Working with GitHub (Remote Repos)

Push your commits to GitHub

■ Upload your work

```
git push origin main # push local main to GitHub
```

origin is just the nickname Git gives to the GitHub URL. **main** is the branch name.

Pull the latest changes from GitHub

Before starting work each day, pull to get your teammate's latest commits:

■ Get teammate's latest work

```
git pull origin main
```

Linking a local repo to GitHub for the first time

Create a new empty repo on GitHub (no README), then link it:

■ First push to GitHub

```
git remote add origin https://github.com/username/repo-name.git
git branch -M main
git push -u origin main
```

■ The **-u** flag sets the upstream link, so from now on you can just type **git push** without specifying origin main.

9. Git Inside VS Code

VS Code has built-in Git support – you barely need to touch the terminal for day-to-day work.

The Source Control panel

Click the **branch icon** in the left sidebar (or press **Ctrl+Shift+G**). You will see:

- **Changes** – modified files (equivalent to git status).
- **Staged Changes** – files you have clicked + on.
- A **message box** at the top to type your commit message.
- A ✓ **Commit** button.

Common VS Code Git actions

Command	What it does
Click + on a file	Stage that file (git add)
Click + on Changes	Stage all files (git add .)
Type message → Commit	Commit staged files (git commit -m)
... menu → Push	Push to GitHub (git push)
... menu → Pull	Pull from GitHub (git pull)
Branch name (bottom bar)	Switch or create branches

■ Install the **GitLens** extension from the VS Code marketplace. It adds powerful history browsing and line-by-line blame views.

Resolving merge conflicts in VS Code

When two people edit the same line, Git marks the conflict in the file. VS Code highlights these with inline buttons: **Accept Current Change**, **Accept Incoming Change**, or **Accept Both**. Click the one you want, save, then stage and commit.

10. Git with Google Colab

Google Colab notebooks run on temporary cloud machines. Git is pre-installed, but the environment resets each session, so you need to re-clone or re-authenticate each time.

Clone your repo in Colab

■ Clone at the start of each session

```
# Run in a Colab code cell
!git clone https://github.com/username/repo-name.git
%cd repo-name
```

Configure identity in Colab

```
!git config --global user.name "Your Name"
!git config --global user.email "you@example.com"
```

Authenticate with a Personal Access Token (PAT)

Colab cannot use your saved password. Use a GitHub Personal Access Token:

1. On GitHub: Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token.
2. Give it **repo** scope.
3. Copy the token (it looks like *ghp_xxxxxxxxxxxx*).

■ Push with PAT

```
# Push using your token (replace placeholders)
!git push https://YOUR_TOKEN@github.com/username/repo-name.git
```

■■ Never hard-code your token in a notebook that you commit! Use Colab's Secrets feature (the ■ icon in the left sidebar) to store it safely.

Save your notebook changes

```
!git add my_notebook.ipynb
!git commit -m "Update analysis in notebook"
!git push origin main
```

11. Fixing Common Mistakes

Undo the last commit (keep the changes)

```
git reset --soft HEAD~1
```

Your files are unchanged; only the commit is undone. You can re-stage and re-commit.

Discard all uncommitted changes to a file

```
git checkout -- filename.py
```

■■ This is permanent – the changes are gone. Use with caution!

Fix the last commit message

```
git commit --amend -m "Corrected commit message"
```

Only do this if you haven't pushed yet.

Remove a file from staging (un stage)

```
git restore --staged filename.py
```

See what changed in a file

```
git diff filename.py # unstaged changes
git diff --staged filename.py # staged changes
```

12. Collaboration & Pull Requests

A **Pull Request (PR)** is GitHub's way of saying 'I have changes on my branch – please review and merge them into main.'

The collaboration workflow

Step 1 – Create a branch for your feature:

```
git checkout -b feature/my-feature
```

Step 2 – Make commits as usual.

Step 3 – Push the branch to GitHub:

```
git push origin feature/my-feature
```

Step 4 – On GitHub.com, click the banner '*Compare & pull request*' that appears after pushing.

Step 5 – Write a description of what you changed and click **Create Pull Request**.

Step 6 – Your teammate reviews, leaves comments, approves, and merges.

■ Always pull from main before creating a new branch – this reduces the chance of merge conflicts.

.gitignore – files Git should ignore

Some files should never be committed: credentials, large datasets, temporary files. Create a **.gitignore** file in the root of your project:

■ .gitignore

```
# .gitignore example
__pycache__/
*.pyc
.env
data/raw/
.ipynb_checkpoints/
*.csv
```

13. Quick-Reference Cheat Sheet

Command	What it does
<code>git init</code>	Initialise a new local repository
<code>git clone <url></code>	Download a remote repo to your machine
<code>git status</code>	Show changed / staged files
<code>git add <file></code>	Stage a specific file
<code>git add .</code>	Stage all changes
<code>git commit -m "msg"</code>	Save a snapshot with a message
<code>git log --oneline</code>	Compact commit history
<code>git diff</code>	Show unstaged changes
<code>git push origin main</code>	Upload commits to GitHub
<code>git pull origin main</code>	Download latest commits from GitHub
<code>git branch</code>	List branches
<code>git checkout -b <name></code>	Create and switch to a new branch
<code>git checkout main</code>	Switch to the main branch
<code>git merge <branch></code>	Merge a branch into current branch
<code>git branch -d <name></code>	Delete a merged branch
<code>git reset --soft HEAD~1</code>	Undo last commit, keep changes staged
<code>git restore --staged <f></code>	Un-stage a file
<code>git checkout -- <file></code>	Discard all changes to a file (careful!)
<code>git stash</code>	Temporarily shelve uncommitted changes
<code>git stash pop</code>	Restore shelved changes

■ Bookmark this page! Once you know these ~20 commands you can handle 99% of everyday Git tasks.

Happy coding! ■ Remember: every expert was once a beginner. Git feels confusing at first, but after a week of daily use it becomes second nature.